SHOWSHELF LAB · FOR THE AFTERNOON TEACHER

Teacher guide: Day 6

Students work through the steps in the Lab on their own. Each step has an explanation, a task, two hints and an answer button, so most of the time they can get unstuck alone. This guide gives you every task, a correct answer and the mistakes students usually make, so you can help quickly.

How to help: first ask them to read the "Not yet" message and the red Console line (if any), then open a hint. Only point at the answer after that. In the Lab, **Help › Teacher view** shows the answer and these notes on every step.

Day 6: Lists of shows Arrays, objects, loops

A real app doesn't show one show: it shows lists. Today you learn how JavaScript keeps many values together (**arrays**), how it describes one thing with named values (**objects**), and how to go through a whole list with a **loop**.

The file **data.js** (a read-only tab above the code box) holds twelve shows, each one an object with a name, genres, a rating and more. You start in the Console, exploring the data. Then you write small functions that pick shows from the list: the best ones, one by its id, the ones that match a search, the ones in a genre.

As before, a helper file, **page.js**, uses your functions to fill in the page. It also scrolls the preview to the newest part you've made.

After the ten steps come thirty-two short **practice exercises** (P1 to P32) in a practice gym: small functions that use the same ideas again, from very easy to harder.

Step 1. A list: arrays ~15 min

Teaches: Arrays: `[a, b, c]`, `list[0]` is the first item, `.length`, `.push()`.

TASK

1. Create `favourites`: an array with the names of three shows you like.
2. Log the first one, and log how many there are.
3. Add a fourth show with `push`. Then log the whole array.

COMMON MISTAKES

- `favourites[1]` for the first item (counting starts at 0).
- Forgetting the commas between items.
- Round brackets instead of square ones: `("Dark", "Fleabag")`.

A CORRECT ANSWER

```
// app.js: lists of shows.
// data.js (a read-only tab) gives you `shows`: an array of 12 show objects.

const favourites = ["Dark", "Fleabag", "The Wire"];
console.log(favourites[0]);
console.log(favourites.length);

favourites.push("Chernobyl");
console.log(favourites);
```

Step 2. One show: objects ~15 min

Teaches: Objects: `{ key: value, ... }`. Read with `obj.key`, change with `obj.key = value`.

TASK

1. Create an object `myShow` for a show you like, with three properties: `name` (text), `year` (the year it started, a number) and `rating` (a number).
2. Log its name.
3. Change the rating to your own score, with a dot and `=`.
4. Log the whole object.

COMMON MISTAKES

- `=` instead of `:` inside the object: `name = "Fleabag"`.
- A missing comma between properties.
- Quotes around numbers: `year: "2016"`.

A CORRECT ANSWER

```
// ... unchanged ...
favourites.push("Chernobyl");
console.log(favourites);

const myShow = {
  name: "Fleabag",
  year: 2016,
  rating: 8.4,
};
console.log(myShow.name);

myShow.rating = 10;
console.log(myShow);
```

Step 3. The twelve shows ~15 min

Teaches: Arrays of objects: `shows[0].name`. The last item is `list[list.length - 1]`.

TASK

Log four things about the data:

1. how many shows there are
2. the name of the first show
3. the rating of the second show
4. the name of the last show, using `shows.length - 1`

COMMON MISTAKES

- `shows[12]` for the last show (it's `shows[11]`).
- `shows.name` (the array has no name; each show has).

NOTE

Encourage students to open the data.js tab and read one show object in full before they start.

A CORRECT ANSWER

```
// ... unchanged ...
myShow.rating = 10;
console.log(myShow);

console.log(shows.length);
console.log(shows[0].name);
console.log(shows[1].rating);
console.log(shows[shows.length - 1].name);
```

Step 4. Go through every show ~15 min

Teaches: `for (const item of list) { ... }` runs the block once per item.

TASK

Write a `for...of` loop through `shows` that logs the name of every show. You'll see twelve new lines in the Console.

If you like, log the rating too: `console.log(show.name, show.rating);`.

COMMON MISTAKES

- `shows.name` inside the loop instead of `show.name`.
- `for (const show in shows)`: `in` gives the index numbers, not the shows.

A CORRECT ANSWER

```
// ... unchanged ...
console.log(shows[1].rating);
console.log(shows[shows.length - 1].name);

for (const show of shows) {
  console.log(show.name);
}
```

Step 5. Keep only the best ~20 min

Teaches: Pick items: start with `[]`, loop, and `push` the ones that pass an `if`.

TASK

Replace the `TODO` comment: if a show's rating is **9 or more**, push the show into `best`.

The Console shows `best.length`. How many shows are rated 9 or more?

Then try it with `8.8` instead of `9`, and change it back.

COMMON MISTAKES

- The `push` outside the `if` (every show is added).
- `const best = [];` inside the loop (it starts empty every round).
- `>` instead of `>=` (the 9.0 shows are left out).

A CORRECT ANSWER

```
// ... unchanged ...
  console.log(show.name);
}

const best = [];
for (const show of shows) {
  if (show.rating >= 9) {
    best.push(show);
  }
}
console.log(best.length);
```

Step 6. filter: the short way ~20 min

Teaches: `list.filter((item) => test)` gives a new array with the items that pass the test.

TASK

Under your code, write `topRated(list)`. It returns the shows in `list` with a rating of **9 or more**, using `filter`.

The Top rated section on the page fills with five cards.

COMMON MISTAKES

- Forgetting `return` before `list.filter`.
- Using `shows` inside the function instead of the parameter `list`.
- Curly brackets after the arrow without a `return`: `(show) => { show.rating >= 9 }` keeps nothing.

A CORRECT ANSWER

```
// ... unchanged ...
}
console.log(best.length);

function topRated(list) {
  return list.filter((show) => show.rating >= 9);
}
```

Step 7. map: one thing from each ~15 min

Teaches: `list.map((item) => newValue)` gives a new array with one result per item.

TASK

1. Write `names(list)` that returns an array with the name of every show in `list`, using `map`.
2. Log `names(shows)` to see the result in the Console.

COMMON MISTAKES

- Using `filter` instead of `map` (you get whole shows back).
- `shows.name` instead of `show.name` inside the arrow function.

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.rating >= 9);
}

function names(list) {
  return list.map((show) => show.name);
}

console.log(names(shows));
```

Step 8. find: one show by its id ~15 min

Teaches: `list.find((item) => test)` gives back the first matching item, or `undefined`.

TASK

1. Write `getById(list, id)` that returns the show in `list` with that `id`, using `find`.
2. Test it: `console.log(getById(shows, 2).name);` should print Breaking Bad. (Without `.name`, you'd see the whole show object.)

page.js uses it for **Pick of the day**: `getById(shows, 7)`.

COMMON MISTAKES

- `show.id = id` (one equals sign) always gives the first show.
- Using `filter` and getting a list with one show in it.
- `getById(shows, "2")`: the id in data.js is a number, and `===` checks the type.

A CORRECT ANSWER

```
// ... unchanged ...

console.log(names(shows));

function getById(list, id) {
  return list.find((show) => show.id === id);
}

console.log(getById(shows, 2).name);
```

Step 9. Search by name ~20 min

Teaches: Search: `trim()`, `toLowerCase()` on both sides, then `includes()`.

TASK

We added the start of `searchByName(list, term)`. Replace the two `TODO` comments:

1. Make `needle`: the `term`, trimmed and in lower case.
2. Return the shows in `list` whose name, in lower case, includes `needle`.

Then type in the search box on the page. Try `office`, `bad` and `THE`. Why does How I Met Your Mother appear for `THE`?

COMMON MISTAKES

- Making only the search word lower case, not the show's name.
- `show.name === needle` instead of `includes` (only exact names match).
- `toLowerCase` without brackets.

NOTE

On Day 7 students write the event code that page.js does here (listening to the search box). Today, the point is the function.

A CORRECT ANSWER

```
// ... unchanged ...

console.log(getById(shows, 2).name);

function searchByName(list, term) {
  const needle = term.trim().toLowerCase();
  return list.filter((show) => show.name.toLowerCase().includes(needle));
}
```

Step 10. Browse by genre ~15 min

Teaches: `array.includes(value)` checks whether a list contains a value.

TASK

Write `getByGenre(list, genre)`. It returns the shows in `list` whose `genres` include `genre`.

Then click the genre buttons on the page. What happens with **Western**?

COMMON MISTAKES

- `show.genre` instead of `show.genres` (the key has an s).
- `show.genres === genre` (an array is never equal to one string).

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.name.toLowerCase().includes(needle));
}

function getByGenre(list, genre) {
  return list.filter((show) => show.genres.includes(genre));
}
```

Practice 1. How many shows? ~5 min

Teaches: Practice: `.length` counts the items in any list.

TASK

Make `countShows(list)` return the number of shows in `list`.

When every row in the preview is green, press **Check**.

COMMON MISTAKES

- `return shows.length` instead of `list.length` (right for `shows`, wrong for the other lists).
- `list.length - 1` (that's the index of the last item, not the count).
- Forgetting `return`: the rows say "Did you forget return?".

NOTE

Practice exercises are independent: a student can skip one and come back to it later. Each one has its own Hint and Answer buttons, like the steps.

A CORRECT ANSWER

```
// P1. How many shows are in a list?
// countShows(list) gives back how many shows the list has.

function countShows(list) {
  return list.length;
}

// Each tryIt line calls your function and shows the result in the preview.
const two = [{ name: "Dark" }, { name: "Fleabag" }];
const none = [];

tryIt(countShows, [shows], 12);
tryIt(countShows, [two], 2);
tryIt(countShows, [none], 0);
```

Practice 2. The first name ~5 min

Teaches: Practice: `list[0].name` is the first item's name.

TASK

Make `firstName(list)` return the **name** of the first show in `list`.

COMMON MISTAKES

- `list[1]` for the first item (counting starts at 0).
- `list.name` (the list has no name; each show has).
- Returning `list[0]`, the whole show, without `.name`.

A CORRECT ANSWER

```
// P2. The name of the first show
// firstName(list) gives back the name of the first show in the list.

function firstName(list) {
  return list[0].name;
}

const trio = [{ name: "Chernobyl" }, { name: "Dark" }, { name: "The Crown" }];

tryIt(firstName, [shows], "Friends");
tryIt(firstName, [trio], "Chernobyl");
```

Practice 3. The last name ~8 min

Teaches: Practice: the last item is `list[list.length - 1]`.

TASK

Make `lastName(list)` return the **name** of the last show in `list`. It must work for lists of any length, so don't write a fixed number like 11.

COMMON MISTAKES

- `list[list.length]`: one past the end, so the show is `undefined` and reading `.name` crashes.
- `list[11]`: right for `shows`, but crashes for the shorter lists.
- `list.length - 1` without the square brackets around it.

A CORRECT ANSWER

```
// P3. The name of the last show
// lastName(list) gives back the name of the last show, for a list of any length.

function lastName(list) {
  return list[list.length - 1].name;
}

const trio = [{ name: "Chernobyl" }, { name: "Dark" }, { name: "The Crown" }];
const one = [{ name: "Fleabag" }];

tryIt(lastName, [shows], "Avatar: The Last Airbender");
tryIt(lastName, [trio], "The Crown");
tryIt(lastName, [one], "Fleabag");
```

Practice 4. A sentence about one show ~8 min

Teaches: Practice: `${show.name}` puts a property into a template literal.

TASK

Make `aboutShow(show)` return a sentence built from the show's `name`, `premiered` and `network`, like this: **Friends started in 1994 on NBC.**

COMMON MISTAKES

- `shows.name` (the list) instead of `show.name` (the parameter): the sentence says *undefined*.
- Quotes instead of backticks, so `${...}` appears as it is.
- A wrong key such as `show.year` (the key is `premiered`).

A CORRECT ANSWER

```
// P4. A sentence about one show
// aboutShow(show) gets ONE show object and gives back a sentence like:
// "Friends started in 1994 on NBC."

function aboutShow(show) {
  return `${show.name} started in ${show.premiered} on ${show.network}.`;
}

tryIt(aboutShow, [shows[0]], "Friends started in 1994 on NBC.");
tryIt(aboutShow, [shows[6]], "Chernobyl started in 2019 on HBO.");
tryIt(aboutShow, [{ name: "Dark", premiered: 2017, network: "Netflix" }], "Dark started in 2017 on
Netflix.");
```

Practice 5. Shows from one network ~8 min

Teaches: Practice: `filter` with a parameter in the test, `show.network === network`.

TASK

Make `fromNetwork(list, network)` return the shows in `list` whose `network` is the same as the parameter `network`. Give back whole shows, not names.

COMMON MISTAKES

- One equals sign, `show.network = network`: every show is kept.
- Curly brackets after the arrow without `return`: nothing is kept.
- `"network"` in quotes, which compares with the word network instead of the parameter.

A CORRECT ANSWER

```
// P5. Shows from one network
// fromNetwork(list, network) gives back the shows (whole objects) from that network.

function fromNetwork(list, network) {
  return list.filter((show) => show.network === network);
}

const mini = [
  { name: "Chernobyl", network: "HBO" },
  { name: "Friends", network: "NBC" },
  { name: "The Wire", network: "HBO" },
];
const none = [];

tryIt(fromNetwork, [mini, "HBO"], [{ name: "Chernobyl", network: "HBO" }, { name: "The Wire",
network: "HBO" }]);
tryIt(fromNetwork, [mini, "NBC"], [{ name: "Friends", network: "NBC" }]);
tryIt(fromNetwork, [mini, "AMC"], []);
tryIt(fromNetwork, [none, "HBO"], []);
```

Practice 6. Names above a rating ~8 min

Teaches: Practice: `filter` to pick, then `map` to take the names.

TASK

Make `topNames(list, minRating)` return the **names** of the shows in `list` with a rating of `minRating` **or more**.

COMMON MISTAKES

- `map` before `filter`: the names have no rating, so nothing is kept.
- `>` instead of `>=`: a show rated exactly `minRating` is left out.
- Returning the filtered shows without the `map` step.

A CORRECT ANSWER

```
// P6. Names of the shows with at least a given rating
// topNames(list, minRating) gives back the NAMES of the shows rated minRating or more.

function topNames(list, minRating) {
  const good = list.filter((show) => show.rating >= minRating);
  return good.map((show) => show.name);
}

const mini = [
  { name: "Fleabag", rating: 8.4 },
  { name: "Chernobyl", rating: 9.1 },
  { name: "Dark", rating: 8.7 },
];
const none = [];

tryIt(topNames, [mini, 8.5], ["Chernobyl", "Dark"]);
tryIt(topNames, [mini, 9.1], ["Chernobyl"]);
tryIt(topNames, [mini, 9.5], []);
tryIt(topNames, [none, 8], []);
tryIt(topNames, [shows, 9], ["Breaking Bad", "Chernobyl", "The Wire", "Band of Brothers", "Avatar:
The Last Airbender"]);
```

Practice 7. The comedies, by name ~8 min

Teaches: Practice: `show.genres.includes("Comedy")` tests a value inside an array.

TASK

Make `comedyNames(list)` return the **names** of the shows in `list` whose genres include `"Comedy"`.

COMMON MISTAKES

- `show.genres === "Comedy"` (an array is never equal to one string).
- `show.genre` without the s: "Cannot read properties of undefined".
- `"comedy"` in small letters: array `includes` checks capitals too.

A CORRECT ANSWER

```
// P7. The names of the comedies
// comedyNames(list) gives back the NAMES of the shows that have "Comedy" in their genres.

function comedyNames(list) {
  const comedies = list.filter((show) => show.genres.includes("Comedy"));
  return comedies.map((show) => show.name);
}

const mini = [
  { name: "Fleabag", genres: ["Comedy", "Drama"] },
  { name: "Dark", genres: ["Drama", "Mystery", "Science-Fiction"] },
];
const none = [];

tryIt(comedyNames, [mini], ["Fleabag"]);
tryIt(comedyNames, [none], []);
tryIt(comedyNames, [shows], ["Friends", "The Office", "How I Met Your Mother", "Fleabag"]);
```

Practice 8. Started before a year ~10 min

Teaches: Practice: pick names with a loop: empty array, `if`, `push(show.name)`, return after the loop.

TASK

Make `startedBefore(list, year)` return the **names** of the shows that premiered **before** `year` (a show that started in that year itself doesn't count). Try the loop way this time.

COMMON MISTAKES

- `<=` instead of `<`: a show from that very year is included.
- `const found = [];` inside the loop: "found is not defined" after the loop.
- `return found;` inside the loop: only the first show is checked.

A CORRECT ANSWER

```
// P8. Shows that started before a year
// startedBefore(list, year) gives back the NAMES of the shows that premiered before that year.
// Try it with a loop, an if and push.

function startedBefore(list, year) {
  const found = [];
  for (const show of list) {
    if (show.premiered < year) {
      found.push(show.name);
    }
  }
  return found;
}

const mini = [
  { name: "Band of Brothers", premiered: 2001 },
  { name: "Dark", premiered: 2017 },
  { name: "Friends", premiered: 1994 },
];
const none = [];

tryIt(startedBefore, [mini, 2010], ["Band of Brothers", "Friends"]);
tryIt(startedBefore, [mini, 2001], ["Friends"]);
tryIt(startedBefore, [mini, 1990], []);
tryIt(startedBefore, [none, 2000], []);
```

Practice 9. Is it on the list? ~10 min

Teaches: Practice: `list.find(...) !== undefined` is true when something matches.

TASK

Make `hasShow(list, title)` return `true` if `list` has a show whose name is `title`, ignoring capital letters, and `false` otherwise.

`"the office"` should find The Office, but `"Office"` should not: no show is called only Office.

COMMON MISTAKES

- Returning the show itself instead of `true` / `false`.
- Making only one side lower case.
- Using `includes` on the name, so "Office" finds The Office.

A CORRECT ANSWER

```
// P9. Is a show on the list?
// hasShow(list, title) gives back true if a show in the list has exactly that name
// (capital letters don't matter), and false if not.

function hasShow(list, title) {
  const wanted = title.toLowerCase();
  const found = list.find((show) => show.name.toLowerCase() === wanted);
  return found !== undefined;
}

const none = [];

tryIt(hasShow, [shows, "Dark"], true);
tryIt(hasShow, [shows, "the office"], true);
tryIt(hasShow, [shows, "BREAKING BAD"], true);
tryIt(hasShow, [shows, "Office"], false);
tryIt(hasShow, [shows, "Lost"], false);
tryIt(hasShow, [none, "Dark"], false);
```

Practice 10. Count the short shows ~10 min

Teaches: Practice: count with `let count = 0;` before the loop and `count = count + 1;` inside the `if`.

TASK

Make `countShort(list)` return how many shows in `list` have episodes of **30 minutes or less** (the key is `runtime`). Use a counter.

COMMON MISTAKES

- `const count = 0;`: "Assignment to constant variable" on the first `+ 1`.
- `let count = 0;` inside the loop, or `return count;` inside the loop.
- `=== 30` instead of `<= 30`: a 25-minute show isn't counted.

NOTE

This is the first exercise with a counter. If a student is lost, act it out: "Start at zero. For each show, if it's short, add one. At the end, say the number."

A CORRECT ANSWER

```
// P10. Count the short shows
// countShort(list) gives back HOW MANY shows have episodes of 30 minutes or less.
// Use a counter: let count = 0; before the loop, add 1 inside the if.

function countShort(list) {
  let count = 0;
  for (const show of list) {
    if (show.runtime <= 30) {
      count = count + 1;
    }
  }
  return count;
}

const mini = [
  { name: "Fleabag", runtime: 30 },
  { name: "Dark", runtime: 60 },
  { name: "Quick Show", runtime: 25 },
];
const none = [];

tryIt(countShort, [mini], 2);
tryIt(countShort, [none], 0);
tryIt(countShort, [shows], 5);
```

Practice 11. Total episodes ~8 min

Teaches: Practice: add up with `let total = 0;` before the loop and `total = total + ...` inside.

TASK

Make `totalEpisodes(list)` return the total number of episodes of all the shows in `list`.

COMMON MISTAKES

- `total = show.episodes` without `total +`: only the last show is left.
- `return total;` inside the loop: only the first show is added.
- `const total`: "Assignment to constant variable".

A CORRECT ANSWER

```
// P11. Total episodes
// totalEpisodes(list) gives back the number of episodes of all the shows together.

function totalEpisodes(list) {
  let total = 0;
  for (const show of list) {
    total = total + show.episodes;
  }
  return total;
}

const mini = [
  { name: "Chernobyl", episodes: 5 },
  { name: "Band of Brothers", episodes: 10 },
  { name: "Fleabag", episodes: 12 },
];
const none = [];

tryIt(totalEpisodes, [mini], 27);
tryIt(totalEpisodes, [none], 0);
tryIt(totalEpisodes, [shows], 1014);
```

Practice 12. The average rating ~12 min

Teaches: Practice: average = total / `list.length`; `Math.round(x * 10) / 10` keeps one decimal.

TASK

Make `averageRating(list)`:

1. return `0` if `list` is empty
2. otherwise, return the average `rating` of the shows, rounded to one decimal

COMMON MISTAKES

- Dividing by 12 instead of `list.length`.
- `Math.round(average)` without the times-10 trick: 9 instead of 8.7.
- No check for an empty list: the function gives `NaN`.

A CORRECT ANSWER

```
// P12. The average rating
// averageRating(list) gives back the average rating, rounded to one decimal.
// An empty list gives 0.

function averageRating(list) {
  if (list.length === 0) {
    return 0;
  }
  let total = 0;
  for (const show of list) {
    total = total + show.rating;
  }
  return Math.round(total / list.length * 10) / 10;
}

const mini = [
  { name: "Chernobyl", rating: 9.1 },
  { name: "Friends", rating: 8.8 },
  { name: "The Crown", rating: 8 },
];
const two = [{ name: "Dark", rating: 8.7 }, { name: "Game of Thrones", rating: 8.9 }];
const none = [];

tryIt(averageRating, [mini], 8.6);
tryIt(averageRating, [two], 8.8);
tryIt(averageRating, [none], 0);
tryIt(averageRating, [shows], 8.7);
```

Practice 13. Every network once ~12 min

Teaches: Practice: no duplicates: push only when `result.includes(value) === false`.

TASK

Make `uniqueNetworks(list)` return a list of the networks of the shows in `list`, each network only once, in the order they first appear.

COMMON MISTAKES

- Pushing the whole show instead of `show.network`.
- Asking `list.includes(...)` (the list you loop through) instead of the new list.
- `=== true` instead of `=== false`: nothing is ever pushed.

NOTE

Students who have seen `!` elsewhere may write `!networks.includes(show.network)`. That's the same as `=== false` and passes.

A CORRECT ANSWER

```
// P13. Every network once
// uniqueNetworks(list) gives back the networks of the shows, without duplicates,
// in the order they first appear.

function uniqueNetworks(list) {
  const networks = [];
  for (const show of list) {
    if (networks.includes(show.network) === false) {
      networks.push(show.network);
    }
  }
  return networks;
}

const mini = [
  { name: "Chernobyl", network: "HBO" },
  { name: "Dark", network: "Netflix" },
  { name: "The Wire", network: "HBO" },
];
const none = [];

tryIt(uniqueNetworks, [mini], ["HBO", "Netflix"]);
tryIt(uniqueNetworks, [none], []);
tryIt(uniqueNetworks, [shows], ["NBC", "AMC", "HBO", "Netflix", "CBS", "BBC Three", "Nickelodeon"]);
```

Practice 14. The most episodes ~12 min

Teaches: Practice: best so far: `let best = list[0];`, replace it inside the loop when an item beats it.

TASK

Make `mostEpisodes(list)` return the **whole show** with the most episodes. For an empty list it gives back `undefined`.

COMMON MISTAKES

- `best = show.episodes` (a number) instead of `best = show` (the whole show).
- Comparing `show.episodes > best` instead of `> best.episodes`.
- `const best`: "Assignment to constant variable".

NOTE

If the last row says `shows[0]`: that's Friends. The starter compares with the first show of data.js because Friends happens to have the most episodes.

A CORRECT ANSWER

```
// P14. The show with the most episodes
// mostEpisodes(list) gives back the WHOLE SHOW with the most episodes.
// An empty list gives undefined.

function mostEpisodes(list) {
  let best = list[0];
  for (const show of list) {
    if (show.episodes > best.episodes) {
      best = show;
    }
  }
  return best;
}

const mini = [
  { name: "Chernobyl", episodes: 5 },
  { name: "The Wire", episodes: 60 },
  { name: "Fleabag", episodes: 12 },
];
const none = [];

tryIt(mostEpisodes, [mini], { name: "The Wire", episodes: 60 });
tryIt(mostEpisodes, [none], undefined);
tryIt(mostEpisodes, [shows], shows[0]);
```

Practice 15. The longest-running show ~12 min

Teaches: Practice: best so far with a calculation, and an early `return` for the empty list.

TASK

Make `longestRunning(list)` return the **name** of the show that ran the most years (`ended` minus `premiered`). For an empty list, return `""`.

COMMON MISTAKES

- Comparing only `show.ended` or only `show.premiered` instead of the difference.
- No check for an empty list: "Cannot read properties of undefined (reading 'name')".
- Returning `best` (the whole show) instead of `best.name`.

A CORRECT ANSWER

```
// P15. The longest-running show
// longestRunning(list) gives back the NAME of the show that ran the most years
// (ended minus premiered). An empty list gives "".

function longestRunning(list) {
  if (list.length === 0) {
    return "";
  }
  let best = list[0];
  for (const show of list) {
    if (show.ended - show.premiered > best.ended - best.premiered) {
      best = show;
    }
  }
  return best.name;
}

const mini = [
  { name: "Band of Brothers", premiered: 2001, ended: 2001 },
  { name: "The Wire", premiered: 2002, ended: 2008 },
  { name: "Dark", premiered: 2017, ended: 2020 },
];
const none = [];

tryIt(longestRunning, [mini], "The Wire");
tryIt(longestRunning, [none], "");
tryIt(longestRunning, [shows], "Friends");
```

Practice 16. Shows per network ~15 min

Teaches: Practice: `obj[key]` reads or sets a property whose name is in a variable.

TASK

Make `countByNetwork(list)` return an object with one key per network and the number of shows on it, like `{ HBO: 2, Netflix: 1 }`. An empty list gives `{}`.

COMMON MISTAKES

- `counts.network` or `counts.show.network` instead of `counts[show.network]`.
- Adding 1 to a key that doesn't exist yet: the count becomes `NaN`.
- Setting every count to 1, without the `else` that adds 1.

NOTE

Square brackets with a variable (`counts[show.network]`) are new here; the steps only used the dot. The extra challenge *Count the genres* uses the same idea, so students who finish this one are ready for it.

A CORRECT ANSWER

```
// P16. How many shows per network?
// countByNetwork(list) gives back an object like { HBO: 2, Netflix: 1 }:
// one key per network, and the number of shows on it.

function countByNetwork(list) {
  const counts = {};
  for (const show of list) {
    if (counts[show.network] === undefined) {
      counts[show.network] = 1;
    } else {
      counts[show.network] = counts[show.network] + 1;
    }
  }
  return counts;
}

const mini = [
  { name: "Chernobyl", network: "HBO" },
  { name: "Dark", network: "Netflix" },
  { name: "The Wire", network: "HBO" },
];
const none = [];

tryIt(countByNetwork, [mini], { HBO: 2, Netflix: 1 });
tryIt(countByNetwork, [none], {});
tryIt(countByNetwork, [shows], { NBC: 2, AMC: 1, HBO: 4, Netflix: 2, CBS: 1, "BBC Three": 1,
Nickelodeon: 1 });
```

Practice 17. One label per show ~8 min

Teaches: Practice: `map` with a template literal makes one text per item.

TASK

Make `labels(list)` return a list with one label per show, like **Friends (NBC, 1994)**: the name, then the network and the year it premiered in round brackets.

COMMON MISTAKES

- Quotes instead of backticks, so `${...}` appears as it is.
- `filter` instead of `map`: you get shows back, not labels.
- `shows.name` inside the arrow instead of `show.name`.

A CORRECT ANSWER

```
// P17. One label per show
// labels(list) gives back a list of labels like "Friends (NBC, 1994)", one per show.

function labels(list) {
  return list.map((show) => `${show.name} (${show.network}, ${show.premiered})`);
}

const mini = [
  { name: "Dark", network: "Netflix", premiered: 2017 },
  { name: "The Wire", network: "HBO", premiered: 2002 },
];
const none = [];

tryIt(labels, [mini], ["Dark (Netflix, 2017)", "The Wire (HBO, 2002)"]);
tryIt(labels, [none], []);
```

Practice 18. How many ended before? ~5 min

Teaches: Practice: `list.filter(...).length` counts the matching items.

TASK

Make `countEndedBefore(list, year)` return **how many** shows in `list` ended **before** `year` (a show that ended in that year itself doesn't count).

COMMON MISTAKES

- Returning the filtered list without `.length`.
- `show.premiered` instead of `show.ended`.
- `<=` instead of `<`.

A CORRECT ANSWER

```
// P18. How many shows ended before a year?
// countEndedBefore(list, year) gives back HOW MANY shows ended before that year.

function countEndedBefore(list, year) {
  return list.filter((show) => show.ended < year).length;
}

const mini = [
  { name: "Friends", ended: 2004 },
  { name: "Dark", ended: 2020 },
  { name: "The Wire", ended: 2008 },
];
const none = [];

tryIt(countEndedBefore, [mini, 2010], 2);
tryIt(countEndedBefore, [mini, 2008], 1);
tryIt(countEndedBefore, [none, 2010], 0);
tryIt(countEndedBefore, [shows, 2010], 4);
```

Practice 19. Which decade? ~8 min

Teaches: Practice: `Math.floor(year / 10) * 10` gives the start of the decade.

TASK

Make `decade(show)` return the decade the show premiered in, as text: **1990s** for Friends (1994), **2010s** for Chernobyl (2019).

COMMON MISTAKES

- `Math.round` instead of `Math.floor`: 2017 becomes the 2020s.
- Forgetting `* 10`: "201s".
- Returning the number 1990 without the `s`.

A CORRECT ANSWER

```
// P19. Which decade did a show start in?
// decade(show) gets ONE show and gives back text like "1990s".

function decade(show) {
  const start = Math.floor(show.premiered / 10) * 10;
  return `${start}s`;
}

tryIt(decade, [shows[0]], "1990s");
tryIt(decade, [shows[6]], "2010s");
tryIt(decade, [{ name: "Band of Brothers", premiered: 2001 }], "2000s");
tryIt(decade, [{ name: "Dark", premiered: 2017 }], "2010s");
```

Practice 20. The rating of a show ~8 min

Teaches: Practice: `if (found === undefined)` before reading `found.rating`.

TASK

Make `ratingOf(list, title)` return the `rating` of the show whose name is `title` (capital letters don't matter). If there's no such show, return `0`.

COMMON MISTAKES

- No `if`: "Cannot read properties of undefined (reading 'rating')" for an unknown name.
- Returning the whole show instead of `found.rating`.
- Making only one side lower case.

A CORRECT ANSWER

```
// P20. The rating of a show, by its name
// ratingOf(list, title) gives back the rating of the show called title
// (capital letters don't matter), or 0 if there is no such show.

function ratingOf(list, title) {
  const wanted = title.toLowerCase();
  const found = list.find((show) => show.name.toLowerCase() === wanted);
  if (found === undefined) {
    return 0;
  }
  return found.rating;
}

const none = [];

tryIt(ratingOf, [shows, "Dark"], 8.7);
tryIt(ratingOf, [shows, "the office"], 8.6);
tryIt(ratingOf, [shows, "CHERNOBYL"], 9.1);
tryIt(ratingOf, [shows, "Lost"], 0);
tryIt(ratingOf, [none, "Dark"], 0);
```

Practice 21. Small objects with map ~10 min

Teaches: Practice: `list.map((x) => ({ key: value }))` makes a list of new objects.

TASK

Make `nameAndYears(list)` return one small object per show, with two properties: `name` (the show's name) and `years` (`ended` minus `premiered`). Friends becomes `{ name: "Friends", years: 10 }`.

COMMON MISTAKES

- No round brackets around the object: every item is `undefined`.
- `premiered - ended` (a negative number of years).
- Keeping the whole show instead of a new object with two properties.

NOTE

The round brackets around an object after `=>` are a real JavaScript quirk. If a student is stuck, show them the longer version with `{ return { ... }; }`, which also passes.

A CORRECT ANSWER

```
// P21. Small objects with map
// nameAndYears(list) gives back one object per show: { name: ..., years: ... },
// where years is ended minus premiered.

function nameAndYears(list) {
  return list.map((show) => ({ name: show.name, years: show.ended - show.premiered }));
}

const mini = [
  { name: "The Wire", premiered: 2002, ended: 2008, network: "HBO" },
  { name: "Chernobyl", premiered: 2019, ended: 2019, network: "HBO" },
];
const none = [];

tryIt(nameAndYears, [mini], [{ name: "The Wire", years: 6 }, { name: "Chernobyl", years: 0 }]);
tryIt(nameAndYears, [none], []);
```

Practice 22. Longer than N years ~8 min

Teaches: Practice: a calculation as the test: `filter((show) => show.ended - show.premiered > n)`.

TASK

Make `longRunners(list, minYears)` return the **names** of the shows that ran **longer than** `minYears` years (`ended` minus `premiered`).

COMMON MISTAKES

- `>=` instead of `>`: shows of exactly `minYears` are included.
- `premiered - ended` (always negative, so nothing is kept).
- Returning the shows without the `map` to names.

A CORRECT ANSWER

```
// P22. Shows that ran longer than N years
// longRunners(list, minYears) gives back the NAMES of the shows with
// (ended minus premiered) greater than minYears.

function longRunners(list, minYears) {
  const long = list.filter((show) => show.ended - show.premiered > minYears);
  return long.map((show) => show.name);
}

const mini = [
  { name: "Friends", premiered: 1994, ended: 2004 },
  { name: "Dark", premiered: 2017, ended: 2020 },
  { name: "The Office", premiered: 2005, ended: 2013 },
];
const none = [];

tryIt(longRunners, [mini, 5], ["Friends", "The Office"]);
tryIt(longRunners, [mini, 8], ["Friends"]);
tryIt(longRunners, [mini, 20], []);
tryIt(longRunners, [none, 1], []);
```

Practice 23. Short and good: two conditions ~10 min

Teaches: Practice: `a && b` is true only when both are true.

TASK

Make `shortAndGood(list, minRating)` return the **names** of the shows with episodes of **30 minutes or less and** a rating of `minRating` **or more**.

COMMON MISTAKES

- Only one of the two tests.
- `show.runtime <= 30 && >= minRating` (the second test needs its own `show.rating`).
- A single `&` instead of `&&`.

NOTE

`&&` is new in this exercise; the explanation introduces it. "And" in plain English is the right way to read it aloud.

A CORRECT ANSWER

```
// P23. Short AND good
// shortAndGood(list, minRating) gives back the NAMES of the shows with
// episodes of 30 minutes or less AND a rating of minRating or more.

function shortAndGood(list, minRating) {
  const picked = list.filter((show) => show.runtime <= 30 && show.rating >= minRating);
  return picked.map((show) => show.name);
}

const mini = [
  { name: "Fleabag", runtime: 30, rating: 8.4 },
  { name: "Dark", runtime: 60, rating: 8.7 },
  { name: "Friends", runtime: 30, rating: 8.8 },
];
const none = [];

tryIt(shortAndGood, [mini, 8.5], ["Friends"]);
tryIt(shortAndGood, [mini, 8], ["Fleabag", "Friends"]);
tryIt(shortAndGood, [none, 8], []);
tryIt(shortAndGood, [shows, 8.5], ["Friends", "The Office", "Avatar: The Last Airbender"]);
```

Practice 24. Started between two years ~10 min

Teaches: Practice: a range: `from <= x && x <= to`.

TASK

Make `startedBetween(list, from, to)` return the **names** of the shows that premiered from the year `from` to the year `to`, **both years included**. Use a loop, an `if` and `push`.

COMMON MISTAKES

- `from <= show.premiered <= to`, written like maths: every show is kept.
- `<` instead of `<=`: the first or last year is left out.
- `const found = [];` inside the loop.

A CORRECT ANSWER

```
// P24. Shows that started between two years
// startedBetween(list, from, to) gives back the NAMES of the shows that
// premiered from the year "from" to the year "to" (both included).
// Use a loop, an if and push.

function startedBetween(list, from, to) {
  const found = [];
  for (const show of list) {
    if (from <= show.premiered && show.premiered <= to) {
      found.push(show.name);
    }
  }
  return found;
}

const mini = [
  { name: "The Office", premiered: 2005 },
  { name: "Breaking Bad", premiered: 2008 },
  { name: "Game of Thrones", premiered: 2011 },
];
const none = [];

tryIt(startedBetween, [mini, 2005, 2010], ["The Office", "Breaking Bad"]);
tryIt(startedBetween, [mini, 2008, 2008], ["Breaking Bad"]);
tryIt(startedBetween, [mini, 1990, 2000], []);
tryIt(startedBetween, [none, 2000, 2020], []);
```

Practice 25. Does every show have a rating? ~8 min

Teaches: Practice: a flag: `let ok = true;`, set it to `false` in the loop when an item fails.

TASK

Make `allRated(list)` return `true` when every show in `list` has a rating, and `false` when at least one has `rating: null`. Use a flag.

COMMON MISTAKES

- An `else` that sets the flag back to `true`, so only the last show counts.
- `return` inside the loop, so only the first show is checked.
- `show.rating === "null"` in quotes (that's the word null, not the value).

A CORRECT ANSWER

```
// P25. Does every show have a rating?
// allRated(list) gives back true if every show has a rating,
// and false if at least one has rating: null.

function allRated(list) {
  let allHave = true;
  for (const show of list) {
    if (show.rating === null) {
      allHave = false;
    }
  }
  return allHave;
}

const oneMissing = [
  { name: "Dark", rating: 8.7 },
  { name: "Show Without Rating", rating: null },
];
const none = [];

tryIt(allRated, [shows], true);
tryIt(allRated, [oneMissing], false);
tryIt(allRated, [none], true);
```

Practice 26. The shortest name ~10 min

Teaches: Practice: smallest so far: replace `best` when `show.name.length < best.name.length`.

TASK

Make `shortestName(list)` return the shortest show name in `list`. If two are equally short, the first one wins. For an empty list, return `""`.

COMMON MISTAKES

- Comparing `show.name < best.name` (alphabetical order, not length).
- `>` instead of `<` (the longest name).
- `<=` instead of `<`: on a tie the last one wins.

A CORRECT ANSWER

```
// P26. The shortest name
// shortestName(list) gives back the shortest show name.
// On a tie the first one wins. An empty list gives "".

function shortestName(list) {
  if (list.length === 0) {
    return "";
  }
  let best = list[0];
  for (const show of list) {
    if (show.name.length < best.name.length) {
      best = show;
    }
  }
  return best.name;
}

const mini = [{ name: "The Crown" }, { name: "Fleabag" }, { name: "Chernobyl" }];
const tie = [{ name: "Friends" }, { name: "Fleabag" }];
const none = [];

tryIt(shortestName, [mini], "Fleabag");
tryIt(shortestName, [tie], "Friends");
tryIt(shortestName, [none], "");
tryIt(shortestName, [shows], "Dark");
```

Practice 27. The lowest rating ~10 min

Teaches: Practice: lowest so far: replace `best` when `show.rating < best.rating`.

TASK

Make `lowestRated(list)` return the **name** of the show with the lowest rating. On a tie, the first one wins. For an empty list, return `""`.

COMMON MISTAKES

- `>` instead of `<` (the highest rating).
- `<=` instead of `<`: on the tie, How I Met Your Mother wins instead of The Crown.
- No empty-list check: a crash on `best.name`.

A CORRECT ANSWER

```
// P27. The lowest rating
// lowestRated(list) gives back the NAME of the show with the lowest rating.
// On a tie the first one wins. An empty list gives "".

function lowestRated(list) {
  if (list.length === 0) {
    return "";
  }
  let best = list[0];
  for (const show of list) {
    if (show.rating < best.rating) {
      best = show;
    }
  }
  return best.name;
}

const mini = [
  { name: "Dark", rating: 8.7 },
  { name: "Fleabag", rating: 8.4 },
  { name: "Chernobyl", rating: 9.1 },
];
const none = [];

tryIt(lowestRated, [mini], "Fleabag");
tryIt(lowestRated, [none], "");
tryIt(lowestRated, [shows], "The Crown");
```

Practice 28. A watchlist from ids (harder) ~12 min

Teaches: Practice: loop through ids, `find` each show, push it if found.

TASK

Make `watchlist(list, ids)` return the **names** of the shows whose ids are in `ids`, in the order of `ids`. Skip ids that no show has.

COMMON MISTAKES

- Looping through `list` instead of `ids`: the names come in the wrong order.
- No `if`: id 99 crashes with "Cannot read properties of undefined".
- `show.id === ids` (the whole list) instead of `show.id === id`.

A CORRECT ANSWER

```
// P28. A watchlist from ids
// watchlist(list, ids) gives back the NAMES of the shows with those ids,
// in the order of ids. Ids that no show has are skipped.

function watchlist(list, ids) {
  const names = [];
  for (const id of ids) {
    const found = list.find((show) => show.id === id);
    if (found !== undefined) {
      names.push(found.name);
    }
  }
  return names;
}

tryIt(watchlist, [shows, [7, 1]], ["Chernobyl", "Friends"]);
tryIt(watchlist, [shows, [2, 99, 4]], ["Breaking Bad", "Game of Thrones"]);
tryIt(watchlist, [shows, []], []);
```

Practice 29. Every genre once (harder) ~12 min

Teaches: Practice: a loop inside a loop: `for (const genre of show.genres)` inside `for (const show of list)`.

TASK

Make `allGenres(list)` return every genre of the shows in `list`, each only once, in the order they first appear.

COMMON MISTAKES

- Pushing `show.genres` (a whole array) instead of each `genre`.
- The `includes` test missing, so Comedy and Drama appear many times.
- `return` between the two closing `}` (after the first show only).

A CORRECT ANSWER

```
// P29. Every genre once
// allGenres(list) gives back every genre of the shows, without duplicates,
// in the order they first appear.

function allGenres(list) {
  const genres = [];
  for (const show of list) {
    for (const genre of show.genres) {
      if (genres.includes(genre) === false) {
        genres.push(genre);
      }
    }
  }
  return genres;
}

const mini = [
  { name: "Friends", genres: ["Comedy", "Romance"] },
  { name: "Fleabag", genres: ["Comedy", "Drama"] },
];
const none = [];

tryIt(allGenres, [mini], ["Comedy", "Romance", "Drama"]);
tryIt(allGenres, [none], []);
```

Practice 30. The best in a genre (harder) ~12 min

Teaches: Practice: `filter` to narrow the list, then "best so far" on the result.

TASK

Make `bestInGenre(list, genre)` return the **name** of the highest-rated show in `list` with that genre. If no show has it, return `""`.

COMMON MISTAKES

- Looping through `list` instead of the filtered list: the best show of any genre.
- No check for an empty result: a crash for Western.
- `show.genres === genre` instead of `show.genres.includes(genre)`.

A CORRECT ANSWER

```
// P30. The best show in a genre
// bestInGenre(list, genre) gives back the NAME of the highest-rated show
// with that genre, or "" if no show has it.

function bestInGenre(list, genre) {
  const inGenre = list.filter((show) => show.genres.includes(genre));
  if (inGenre.length === 0) {
    return "";
  }
  let best = inGenre[0];
  for (const show of inGenre) {
    if (show.rating > best.rating) {
      best = show;
    }
  }
  return best.name;
}

const mini = [
  { name: "Dark", rating: 8.7, genres: ["Drama", "Mystery"] },
  { name: "Fleabag", rating: 8.4, genres: ["Comedy", "Drama"] },
  { name: "Friends", rating: 8.8, genres: ["Comedy", "Romance"] },
];

tryIt(bestInGenre, [mini, "Comedy"], "Friends");
tryIt(bestInGenre, [mini, "Drama"], "Dark");
tryIt(bestInGenre, [mini, "Western"], "");
tryIt(bestInGenre, [shows, "Fantasy"], "Avatar: The Last Airbender");
```

Practice 31. Rating bands (harder) ~12 min

Teaches: Practice: counters in one object: `{ a: 0, b: 0 }`, then `obj.a = obj.a + 1` in the right branch.

TASK

Make `ratingBands(list)` return an object `{ mustWatch: ..., good: ..., ok: ... }` that counts the shows rated **9 or more** (`mustWatch`), **8 or more** but under 9 (`good`), and **under 8** (`ok`). An empty list gives all three at 0.

COMMON MISTAKES

- `>= 8` tested before `>= 9` : every show lands in `good` .
- `>` instead of `>=` : shows rated exactly 9.0 count as good.
- Creating the object inside the loop.

A CORRECT ANSWER

```
// P31. Rating bands
// ratingBands(list) counts the shows in three bands and gives back
// { mustWatch: ..., good: ..., ok: ... }:
// mustWatch = rated 9 or more, good = 8 or more, ok = under 8.

function ratingBands(list) {
  const bands = { mustWatch: 0, good: 0, ok: 0 };
  for (const show of list) {
    if (show.rating >= 9) {
      bands.mustWatch = bands.mustWatch + 1;
    } else if (show.rating >= 8) {
      bands.good = bands.good + 1;
    } else {
      bands.ok = bands.ok + 1;
    }
  }
  return bands;
}

const mini = [
  { name: "Chernobyl", rating: 9.1 },
  { name: "Dark", rating: 8.7 },
  { name: "Fleabag", rating: 8.4 },
];
const none = [];

tryIt(ratingBands, [mini], { mustWatch: 1, good: 2, ok: 0 });
tryIt(ratingBands, [none], { mustWatch: 0, good: 0, ok: 0 });
tryIt(ratingBands, [shows], { mustWatch: 5, good: 7, ok: 0 });
```

Practice 32. Names per network (harder) ~15 min

Teaches: Practice: group into an object of arrays: create `[]` the first time, then `push`.

TASK

Make `namesByNetwork(list)` return an object with one key per network, whose value is the list of the names of the shows on that network, in the order of `list`. An empty list gives `{}`.

COMMON MISTAKES

- `groups[show.network] = [show.name]` every time: only the last name per network is left.
- `push` before the empty list exists: "Cannot read properties of undefined (reading 'push')".
- `groups.network` instead of `groups[show.network]`.

A CORRECT ANSWER

```
// P32. Names per network
// namesByNetwork(list) gives back an object like
// { HBO: ["Chernobyl", "The Wire"], Netflix: ["Dark"] }

function namesByNetwork(list) {
  const groups = {};
  for (const show of list) {
    if (groups[show.network] === undefined) {
      groups[show.network] = [];
    }
    groups[show.network].push(show.name);
  }
  return groups;
}

const mini = [
  { name: "Chernobyl", network: "HBO" },
  { name: "Dark", network: "Netflix" },
  { name: "The Wire", network: "HBO" },
];
const none = [];

tryIt(namesByNetwork, [mini], { HBO: ["Chernobyl", "The Wire"], Netflix: ["Dark"] });
tryIt(namesByNetwork, [none], {});
```

Extra 1. Best first ~20 min

Teaches: `[...list].sort((a, b) => b.x - a.x)` sorts a copy, biggest first.

TASK

Write `sortByRating(list)` that returns a **copy** of the list, highest rating first. A new section, **Best first**, appears on the page.

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.genres.includes(genre));
}

function sortByRating(list) {
  return [...list].sort((a, b) => b.rating - a.rating);
}
```

Extra 2. Count the genres ~20 min

Teaches: `obj[key]` reads or sets a property whose name is in a variable.

TASK

Write `countByGenre(list)` that returns an object like `{ Comedy: 4, Drama: 8, ... }`. Use loops and an object, not a new array method.

When it works, the genre buttons on the page show their counts.

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.genres.includes(genre));
}

function countByGenre(list) {
  const counts = {};
  for (const show of list) {
    for (const genre of show.genres) {
      if (counts[genre]) {
        counts[genre] = counts[genre] + 1;
      } else {
        counts[genre] = 1;
      }
    }
  }
  return counts;
}
```

Extra 3. Watch everything ~15 min

Teaches: Add up a list: `let total = 0;` before the loop, `total = total + ...` inside.

TASK

Write `totalWatchTime(list)` that returns the total minutes needed to watch every episode of every show in the list (each show: episodes times runtime).

A new section, **Watch everything**, appears on the page.

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.genres.includes(genre));
}

function totalWatchTime(list) {
  let total = 0;
  for (const show of list) {
    total = total + show.episodes * show.runtime;
  }
  return total;
}
```